

Hybrid EWMA–GARCH Daily Volatility Forecaster (PyTorch Port)

Model Research Report (Sample spec, PyTorch) — model_type: TimeSeries — a learnable blend of an exponentially-weighted moving-average variance and a GARCH(1,1) conditional variance

1. Objective and model card

Risk desks forecast daily volatility from a trailing window of log returns. Two classic estimators are the exponentially-weighted moving average (EWMA, RiskMetrics-style, decay lambda) and GARCH(1,1). Each has weaknesses: EWMA has no mean reversion, GARCH is sensitive to initialization. This report specifies a LEARNABLE hybrid that blends the two through a gate w , with all coefficients (lambda, omega, alpha, beta, w) as trainable, reparameterized scalars so they stay in valid ranges under any gradient update. Model card:

```
model_name: HybridVolatilityModel
model_type: TimeSeries
description: EWMA-GARCH hybrid daily variance forecaster. Reads the target series from channel 0 of the
input window, iterates an EWMA variance and a GARCH(1,1) conditional variance over the window, blends
them with a learned gate, and returns the annualized volatility forecast as (batch_size, 1).
hyperparameters: annualization factor = 252; eps = 1e-8
training_hyperparameters: n_epochs = 400; lr = 1e-2; early_stop = 50; batch_size = full-batch;
weight_decay = 0
```

2. Integration contract (mandatory)

The deliverable is ONE Python file named model.py containing exactly one class subclassing torch.nn.Module and the module-level assignment model_cls = <YourClass>. The automated runner applies this contract verbatim:

```
import torch
from model import model_cls
m = model_cls(num_features=10, num_timesteps=4)
for _, param in m.named_parameters():
    param.data.fill_(1.0)
out = m(torch.full((32, 4, 10), 1.0))
assert out.shape == (32, 1) and torch.isfinite(out).all()
```

Consequences: (a) __init__(self, num_features, num_timesteps, **kwargs) with defaults for everything else; (b) forward receives shape (batch_size, num_timesteps, num_features) and returns (batch_size, 1) - do NOT permute the input, timesteps are already axis 1; (c) with every nn.Parameter overwritten by 1.0 and a constant all-ones input the output must stay finite: no division by parameters, no BatchNorm (zero-variance batch gives NaN), prefer LayerNorm with eps or tanh/sigmoid gates; (d) fixed constants belong in register_buffer (buffers are NOT overwritten); (e) no side effects at import time, no try-except, no top-level main; the demo of this report must live inside an if __name__ == '__main__': guard. Only torch and the Python standard library may be imported. model_type: TimeSeries.

3. Model formulation

forward(x) with x of shape (B, T, F). Channel 0 carries the daily log-return series $r = x[0, :, :]$ of shape (B, T). The other channels may be ignored. Define reparameterized scalars (all nn.Parameter, one element each): $\lambda = \text{sigmoid}(p_\lambda)$, $\alpha = \text{sigmoid}(p_\alpha)$, $\beta = \text{sigmoid}(p_\beta)$, $w = \text{sigmoid}(p_w)$, $\omega = \text{abs}(p_\omega)$. Squash the gate inputs into valid ranges so the recurrence is stable for ANY parameter value, including the acceptance test's all-ones overwrite.

Iterate $t = 1..T-1$ over the window using the squared returns $r_2 = r[:, :-1] ** 2$:

EWMA: $v_t = \lambda * v_{t-1} + (1 - \lambda) * r_{2,t}$

GARCH: $s_{2,t} = \omega + \alpha * r_{2,t} + \beta * s_{2,t-1}$

Initialize v_0 to the mean of r_2 (detached, i.e. via .item()) and $s_{2,0}$ to the same scalar. Blend the final-step variances $V = w * v_T + (1 - w) * s_{2,T}$ and return the annualized volatility $\sqrt{V * 252 + 1e-8}$ reshaped to (B, 1). When $T < 2$ there is no recurrence; return $\sqrt{1e-4 * 252 + 1e-8}$ as a (B, 1) constant.

4. Stability requirements

With every parameter overwritten by 1.0 and an all-ones input: $\lambda = \alpha = \beta = w = \text{sigmoid}(1) \sim 0.731$, $\omega = \text{abs}(1) = 1$, and $r^2 = 1$ everywhere, so both recurrences are bounded positive scalars and the output is finite. Because the gate inputs pass through sigmoid/abs, no gradient step can push $\lambda/\alpha/\beta/w$ outside (0,1) or ω below 0, which keeps the variance recursions non-negative and finite. No BatchNorm; no division by a parameter; the only division-free nonlinearity is sqrt of a strictly positive quantity (eps-guarded).

5. Demo data and evaluation (___main___ guard only)

Inside the guard simulate a GARCH(1,1) return stream with `torch.manual_seed(42)`: $T_{\text{total}} = 2500$ daily returns with $\alpha = 0.08$, $\beta = 0.90$ and a long-run variance ω that LEVEL-SHIFTS across three regimes (e.g. $5e-6$ for days 0-799, $4e-5$ for days 800-1699, $1e-5$ thereafter). This non-stationarity is deliberate: a single static baseline cannot track it, so an adaptive forecaster is rewarded. Slide windows with look-back 60 and 5 lagged return channels (construct the model with `num_timesteps = 60`, `num_features = 5`); the label is the realized annualized volatility of the NEXT 5 days, $\sqrt{\text{mean}(r_{\text{next}5}^2) * 252}$. Chronological 70/15/15 split. Train with full-batch Adam (lr $1e-2$, max 400 epochs, early-stop patience 50 on validation MSE, restore best weights). Report test MSE for the model and for two static baselines: (a) the unconditional variance computed from the whole-sample mean ω , and (b) the training-mean realized volatility held constant. Reference run: model ~ 0.016 , beating the constant baseline ~ 0.024 and the unconditional baseline ~ 0.044 . Success: model test MSE is lower than BOTH static baselines.

6. Reference

J.P. Morgan/Reuters. RiskMetrics - Technical Document (4th ed.), 1996 (EWMA volatility). T. Bollerslev. Generalized Autoregressive Conditional Heteroskedasticity. Journal of Econometrics 31, 1986 (GARCH).